

Chapter 2: Bayesian Computation

Exercise 1: Non-Markovian methods: Grid computing

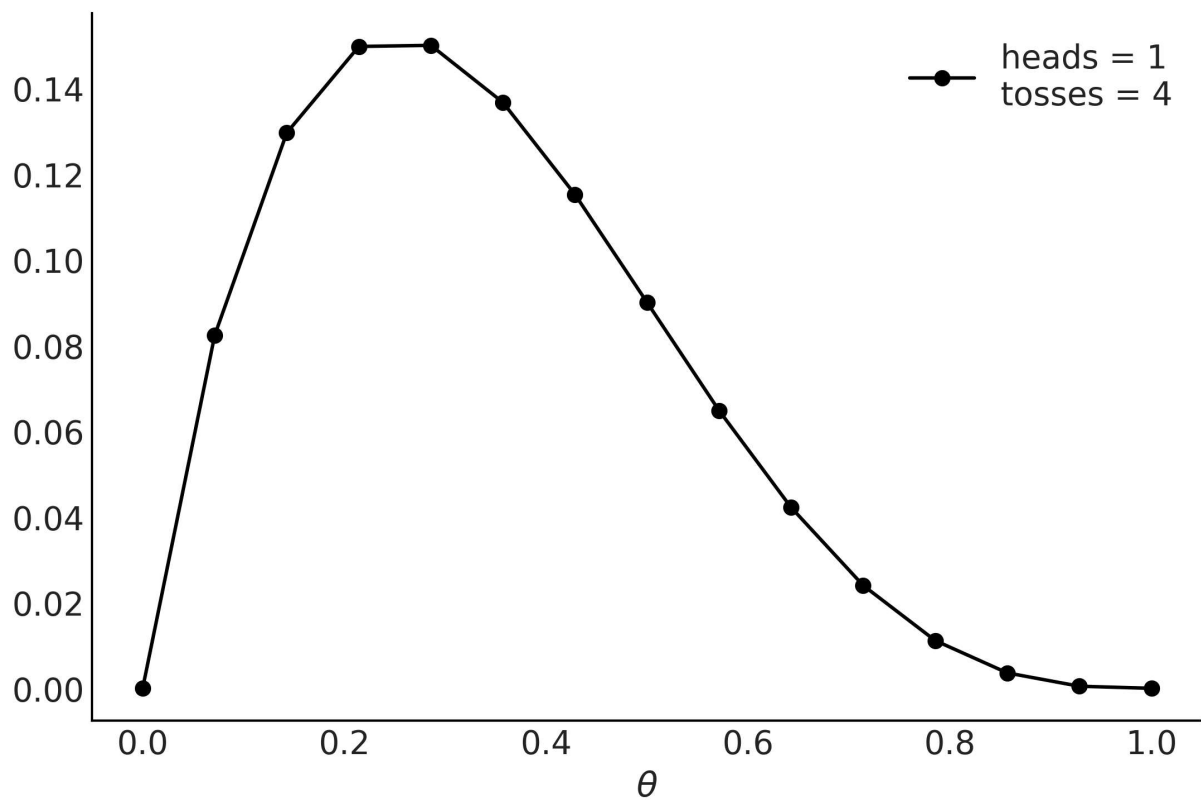
```
In [48]: import numpy as np
import matplotlib.pyplot as plt
from scipy import stats
import pymc as pm
```

```
In [49]: def posterior_grid(grid_points=100, heads=6, tosses=9):
    grid = np.linspace(0, 1, grid_points)
    prior = np.repeat(5, grid_points)
    likelihood = stats.binom.pmf(heads, tosses, grid)
    unstd_posterior = likelihood * prior
    posterior = unstd_posterior / unstd_posterior.sum()
    return grid, posterior

points = 15
h, n = 1, 4

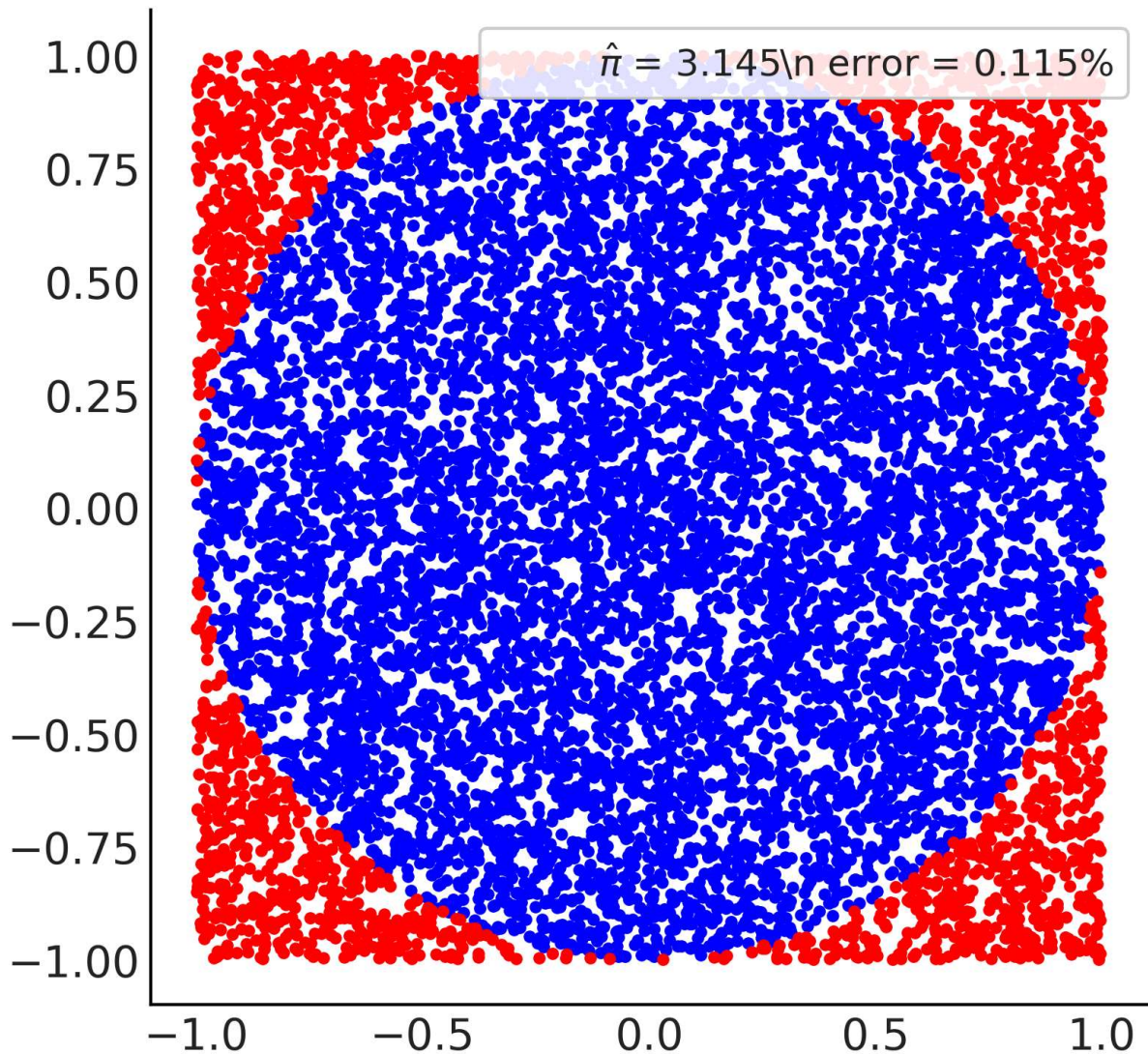
grid, posterior = posterior_grid(points, h, n)

plt.plot(grid, posterior, 'o-', label=f'heads = {h}\ntosses = {n}')
plt.xlabel(r'$\theta$')
plt.legend(loc=0)
plt.show()
```



Exercise 2: Monte Carlo

```
In [62]: N = 10000
x, y = np.random.uniform(-1, 1, size=(2, N))
inside = (x**2 + y**2) <= 1
pi = inside.sum()*4/N
error = abs((pi - np.pi)/pi)* 100
outside = np.invert(inside)
plt.plot(x[inside], y[inside], 'b.')
plt.plot(x[outside], y[outside], 'r.')
plt.plot(0, 0,
        label=r'$\hat{\pi} = {:.4f}$\n error = {:.4f}%'.format(pi, error),
        alpha=0)
plt.axis('square')
plt.legend(frameon=True, framealpha=0.9, fontsize=12);
```



Exercise 3 : Metropolis-Hastings

The following code shows a very simple implementation of the Metropolis algorithm. It is not designed to solve a real problem, but to demonstrate that we can generate samples from a function as long as we can compute its value at a given point. This example is not Bayesian, since it does not use prior information or data. It highlights that MCMC methods are general and can be applied in many different areas. For example, in fields like physics, instead of working with probabilities, we might use a function that measures the energy of a system.

In this code, the input to the metropolis function is a SciPy distribution. We assume that we cannot directly generate random samples from it. However, we can still compute the height (probability value) of the distribution at any point using `pdf()`.

We are NOT directly getting random samples.

Instead we are only checking: "Is this point high or low on the curve?"

Why this is enough for Metropolis

Metropolis only needs to compare two points:

Is x_{i+1} better than x_i

We check:

$\text{pdf}(x_{i+1})$ vs $\text{pdf}(x_i)$

So even without knowing the full distribution, we can compare probabilities locally decide whether to move or stay

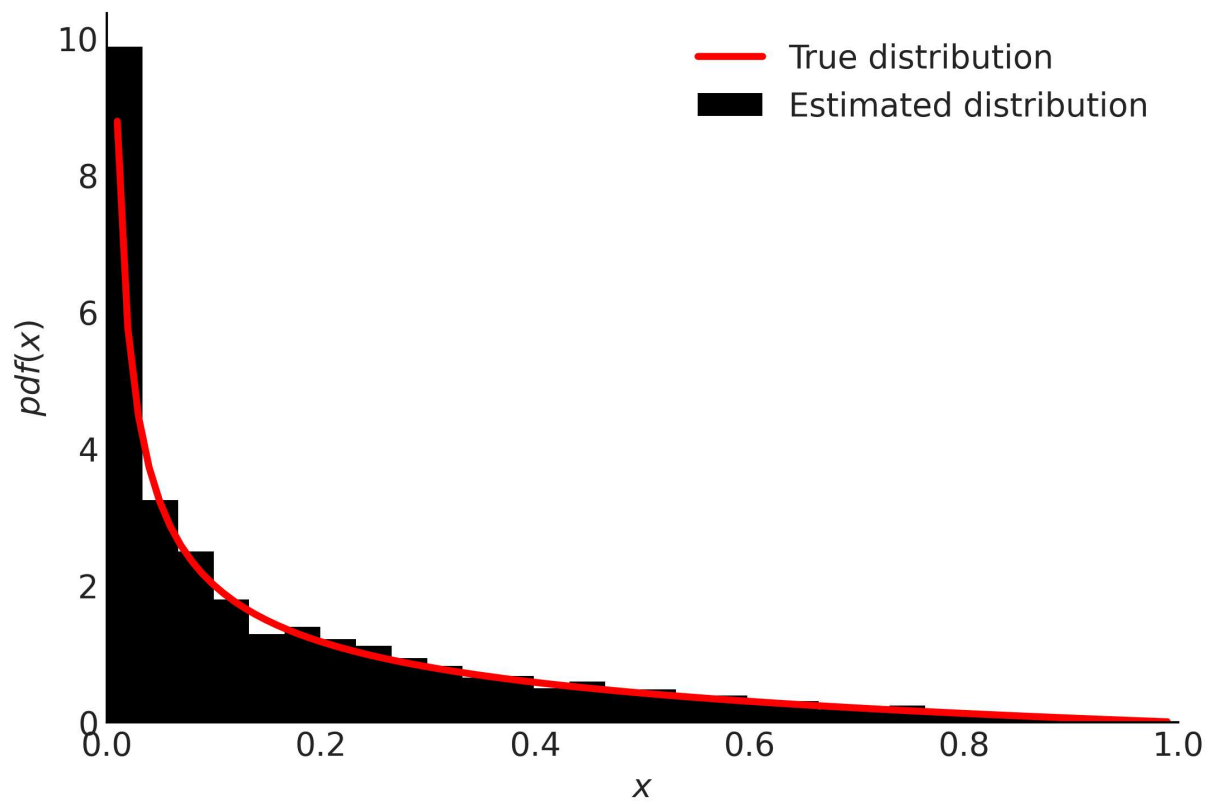
```
In [63]: def metropolis(func, steps=10000):
          """A very simple Metropolis implementation"""
          samples = np.zeros(steps)
          old_x = func.mean()
          old_prob = func.pdf(old_x)
          for i in range(steps):
              new_x = old_x + np.random.normal(0, 0.5)
              new_prob = func.pdf(new_x)
              acceptance = new_prob/old_prob
              if acceptance >= np.random.random():
                  samples[i] = new_x
                  old_x = new_x
                  old_prob = new_prob
              else:
                  samples[i] = old_x
          return samples
```

We use the Beta distribution only as a known reference so that we can compare it with the samples generated by the Metropolis algorithm. The Beta pdf() gives the exact true distribution, while the Metropolis algorithm produces an approximation using samples. By plotting both together, we can visually check how closely the sampled values match the true distribution.

```
In [64]: func = stats.beta(0.4, 2)
          samples = metropolis(func=func)

          x = np.linspace(0.01, .99, 100)
          y = func.pdf(x)

          plt.xlim(0, 1)
          plt.plot(x, y, 'r-', lw=3, label='True distribution')
          plt.hist(samples, bins=30, density=True, label='Estimated distribution')
          plt.xlabel('$x$', fontsize=14)
          plt.ylabel('$pdf(x)$', fontsize=14)
          plt.legend(fontsize=14)
          plt.show()
```



In []: